Introduction to Electronic Design Automation
National Taiwan University
Spring 2026

Problem Set 1
Reference Solution

Last updated: April 23, 2026. Fix a typo in Problem 5(c). ($x \leq 1 \rightarrow x \geq 1$)

1 [Design Methodology and Design Flow]

(a) (4%)

- Modeling: brain seizure signal, electrode-brain interaction
- Design: hardware implants, power supply system, seizure detection algorithm, user interface
- Analysis: safety, reliability, durability, clinical validation, manufacturing cost

(b) (4%)

For real-time, high throughput computational tasks, use dedicated hardware for speed and energy efficiency: image pre-processing, convolutional layers, activation functions, pooling layers.

For flexible, memory-bound or low computation tasks, use software for saving resources and the ease of modification: fully connected layers (if the feature size is large), output (softmax+classification).

(c) (4%)

The synthesis tasks that convert a system specification to a physical layout include *logic synthesis* and *physical synthesis*. In logic synthesis, the design is typically abstracted as RTL representations or gate-level representations. In physical synthesis, the design is typically abstracted as a graph with annotated physical properties, including timing, power, area, and positions.

2 [Algorithm and Complexity]

(a) (2% complexity, 2% #multiplications, 2% #additions)

$$\begin{aligned} \#\text{multiplications} &= \sum_{i=1}^{n} i = \frac{1}{2} n(n+1) \\ \#\text{additions} &= n \\ \text{time complexity} &\in \Theta(n^2) \end{aligned}$$

(b) (2% complexity, 2% #multiplications, 2% #additions)

$$\begin{aligned} \#\text{multiplications} &= n \\ \#\text{additions} &= n \\ \text{time complexity} &\in \Theta(n) \end{aligned}$$

3 [NP-Completeness and Polynomial-Time Reduction]

To prove a decision problem is NP-complete, you need to:

1. Show that the problem is in NP.
2. Give a polynomial time reduction from another problem, which is already proven to be NP-hard.
3. Show that two instances have the same answer.

(a) (4% membership, 6% hardness)

We shall show that the feasibility problem of 0,1-ILP constraints is in NP.

We are given a set of 0,1-ILP constraints and an assignment to the 0,1-variables. To check if this assignment is a feasible solution, it suffices to check, for each 0,1-ILP constraint, if the assignment satisfies the constraint. This can be done in linear time to the size of the constraints. Therefore, if there are n constraints and the size of each constraint is at most m, the feasibility check can be done in $O(mn)$.

We shall then provide a polynomial time reduction from SAT to 0,1-ILP feasibility problem. Let the CNF formula of the SAT instance be

$$\phi = \bigwedge_{k=1}^{m} C_i,$$

where m is the number of clauses, C_i is a clause, and we assume that there are n variables $\{v_1, \ldots, v_n\}$. For a literal l, we use $\text{var}(l)$ to represent its corresponding variable in the instance. We construct the 0,1-ILP feasibility instance as follows:

1. Create n 0,1-variables, say $\{x_1, \ldots, x_n\}$.
2. For each clause C_i, create a 0,1-ILP constraint L_i. Suppose that the number of literals in C_i is k and C_{ij} is the j-th literal of C_i.

$$L_i = \sum_{j=1}^{k} \text{vmap}(C_{ij}) \geq 1,$$

where

$$\text{vmap}(l) = \begin{cases} 1 - x_i & \text{if } l = \neg v_i \\ x_i & \text{if } l = x_i \end{cases}$$

with $\text{var}(l) = v_i$. We remark that L_i is in general not a 0,1-ILP constraint since it may have constants on its LHS. It can be made an equivalent 0,1-ILP constraint through transpositions of those constant terms.

It is a polynomial reduction since the size of the 0,1-ILP feasibility instance is the same as that of the SAT instance. We shall now show that the SAT instance has a solution if and only if the 0,1-ILP feasibility instance has a solution. To distinguish between integer and Boolean values, here we use $\top$ for the Boolean value TRUE and $\bot$ for the Boolean value FALSE.

(if): If the 0,1-ILP feasibility instance has a solution, we can construct the corresponding assignment in the SAT instance, such that $x_i = \top$ if $v_i = 1$ and $x_i = \bot$ if $v_i = 0$. Also, it can be checked that $\text{vmap}(l) = 1$ if the literal l is true under an assignment to v_i. When a linear constraint is true, it means that there is at least one literal with value $\top$ in the corresponding clause, indicating that the clause is satisfied. Therefore, since the 0,1-ILP feasibility solution satisfies all the linear constraints, the constructed assignment in the SAT instance satisfies all the clauses, and thus is a solution.

(only if): If the SAT instance has a solution, we can construct the corresponding assignment in the 0,1-ILP instance, such that $v_i = 1$ if $x_i = \top$ and $v_i = 0$ if $x_i = \bot$. Similarly, it can be checked that the literal l is true iff $\text{vmap}(l) = 1$ under an assignment to x_i. When a clause is satisfied, it means that there is at least one $\text{vmap}(l)$ which is equal to 1 in the corresponding 0,1-ILP constraint, indicating that the constraint is satisfied. Therefore, since the SAT solution satisfies all clauses, the constructed assignment in the 0,1-ILP feasibility instance satisfies all the 0,1-ILP constraints, and thus is a solution.

(b) (4% membership, 6% hardness)

We shall show that the Circuit-SAT problem is in NP.

We are given a circuit consisting of AND, OR, NOT gates and an assignment to the primary inputs. To check if this assignment makes the circuit output true, evaluate the output of each logic gate in topological order. This can be done in linear time to the size of the circuit.

We shall then provide a polynomial time reduction from 0,1-ILP feasibility problem to Circuit-SAT. We assume that there are n 0,1-variables $\{x_1, \ldots, x_n\}$ and m constraints. We construct the circuit as follows:

1. Create n primary inputs, representing the values of $\{x_1, \ldots, x_n\}$.
2. Suppose that the number of terms in a constraint L_i is k and L_{ij} is the j-th variable in L_i. For each clause $\sum_{j=1}^{k} a_{ij} L_{ij} \geq b_i$, compute $a_{ij} L_{ij}$ by performing AND with the corresponding primary input of L_{ij} for each bit of a_{ij}. Since a_{ij} has a fixed binary representation, this does not require any gate.
3. Create cascade of adders computing $-b_i + \sum_{j=1}^{k} a_{ij} L_{ij}$. This requires adding gates linear to k times the length of the coefficients for each clause.
4. Create a NOR gate combining the sign bits of the previous step. This requires $m - 1$ OR gates and a NOT gate. This is the output of the circuit.

It is a polynomial reduction since the size of the circuit is quadratic to the size of the 0,1-ILP feasibility instance.

We shall now show that the 0,1-ILP feasibility instance has a solution if and only if the Circuit-SAT instance has a solution. The circuit output is true if and only if none of the sign bits are true. This is equivalent to having $-b_i + \sum_{j=1}^{k} a_{ij} L_{ij} \geq 0$ for every constraint L_i. Therefore, the satisfying assignment of the circuit is exactly a 0,1-ILP feasibility solution, and vice versa.

4 [SAT and Integer Linear Programming]

(a) (4% formulation, 4% solving)

Let $x_{ij} = 1$ denotes coloring vertex i with color j. The SAT formulation is as follows:

$$\bigwedge_{i=1}^{7} \text{colored}(i) \wedge \text{diff}(1,2) \wedge \text{diff}(2,3) \wedge \text{diff}(3,4) \wedge \text{diff}(4,5) \\ \wedge \text{diff}(5,1) \wedge \text{diff}(1,6) \wedge \text{diff}(3,6) \wedge \text{diff}(2,7) \wedge \text{diff}(4,7),$$

where $\text{colored}(i) = (x_{i1} \vee x_{i2}) \wedge (\neg x_{i1} \vee \neg x_{i2})$ and $\text{diff}(m,n) = (\neg x_{m1} \vee \neg x_{n1}) \wedge (\neg x_{m2} \vee \neg x_{n2})$ if $k = 2$;
$\text{colored}(i) = (x_{i1} \vee x_{i2} \vee x_{i3}) \wedge (\neg x_{i1} \vee \neg x_{i2}) \wedge (\neg x_{i1} \vee \neg x_{i3}) \wedge (\neg x_{i2} \vee \neg x_{i3})$ and $\text{diff}(m,n) = (\neg x_{m1} \vee \neg x_{n1}) \wedge (\neg x_{m2} \vee \neg x_{n2}) \wedge (\neg x_{m3} \vee \neg x_{n3})$ if $k = 3$.

$k = 2$ is unsatisfiable, while $k = 3$ is satisfiable.

(b) (4% formulation, 4% solving)

Let $x_{ij} \in \{0,1\}$ and $x_{ij} = 1$ denotes coloring vertex i with color j. The ILP formulation is as follows:

$$\begin{aligned} &\min / \\ &\text{s.t. } \text{colored}(i)\ \forall i \in \{1,2,\ldots,7\} \\ &\quad \text{diff}(1,2) \\ &\quad \text{diff}(2,3) \\ &\quad \text{diff}(3,4) \\ &\quad \text{diff}(4,5) \\ &\quad \text{diff}(5,1) \\ &\quad \text{diff}(1,6) \\ &\quad \text{diff}(3,6) \\ &\quad \text{diff}(2,7) \\ &\quad \text{diff}(4,7) \end{aligned}$$

where $\text{colored}(i) = \boxed{x_{i1} + x_{i2} = 1}$ and $\text{diff}(m,n) = \boxed{\begin{array}{l} x_{m1} + x_{n1} \leq 1 \\ x_{m2} + x_{n2} \leq 1 \end{array}}$ if $k = 2$;

$\text{colored}(i) = \boxed{x_{i1} + x_{i2} + x_{i3} = 1}$ and $\text{diff}(m,n) = \boxed{\begin{array}{l} x_{m1} + x_{n1} \leq 1 \\ x_{m2} + x_{n2} \leq 1 \\ x_{m3} + x_{n3} \leq 1 \end{array}}$ if $k = 3$.

$k = 2$ is infeasible, while $k = 3$ is feasible.

5 [Model of Computation]

(a) (5%, only non-deterministic: 2%, otherwise 0%)

The finite automaton is shown in Fig. 1.

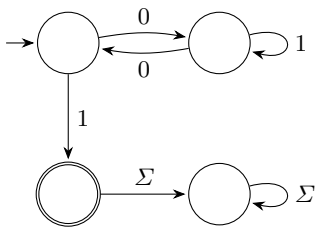

Fig. 1. Finite automaton for $(01^*0)^*1$

(b) (5%)

The regular expression is $0^*11^*0(11^*0)^*(00^*11^*0(11^*0)^*)^*$.

(c) (5%)

The timed automaton is shown in Fig. 2.

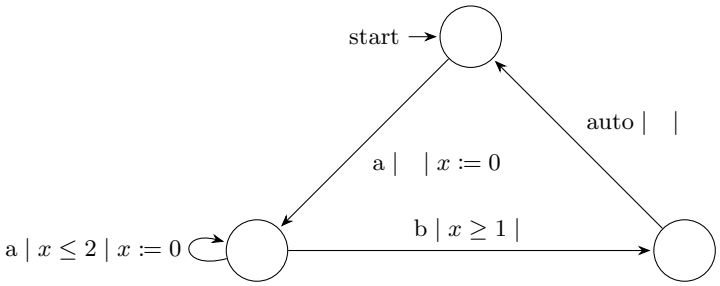

Fig. 2. Timed automaton for timed access control system

(d) (5%)

The Petri net is shown in Fig. 3. The actions shift1 and shift2 are executed automatically by the buffer. All other actions are executed by the corresponding producer/consumer.

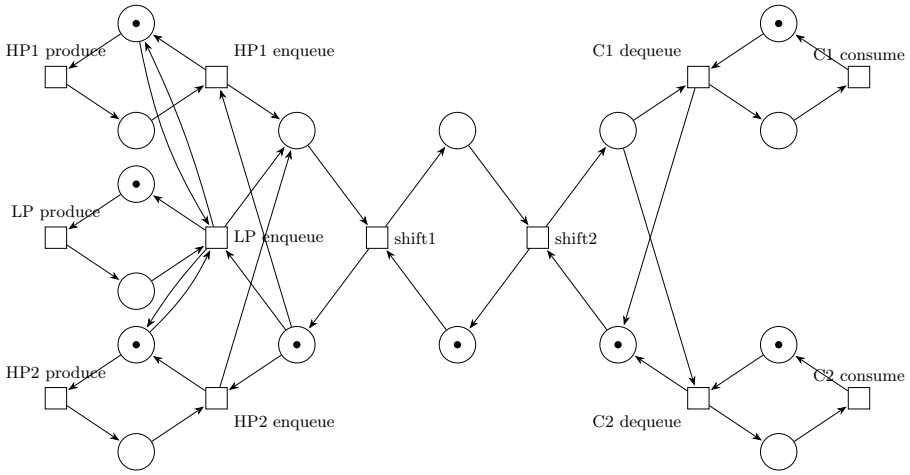

Fig. 3. Petri net for the producer-consumer system

6 [Scheduling]

(a) (4%)

The DFG is shown in Fig. 4

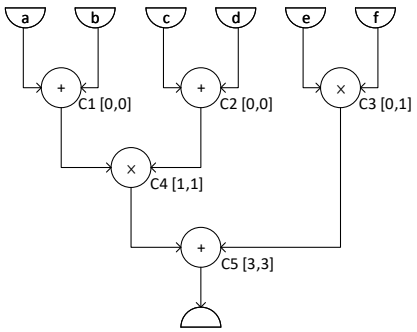

Fig. 4. The DFG in problem 6(a) with start time of each node in [ASAP 6(b), ALAP 6(c)] scheduling respectively.

(b) (4%)

The start times of each node with [ASAP, ALAP] scheduling are shown in figure Fig. 4. Two adders and two multipliers are required for ASAP scheduling. The time chart of the scheduling result is shown in Fig. 5

		0	1	2	3	4
Adder	#1	C1			C5	
Adder	#2	C2				
Multiplier	#1	C3				
Multiplier	#2		C4			

Fig. 5. The time chart of the scheduling result in problem 6(b).

(c) (4%)

Two adders and two multipliers are required for ALAP scheduling. The time chart of the scheduling result is shown in Fig. 6

		0	1	2	3	4
Adder	#1	C1			C5	
Adder	#2	C2				
Multiplier	#1		C3			
Multiplier	#2		C4			

Fig. 6. The time chart of the scheduling result in problem 6(c).

(d) (4%)

Since there are only two possible mobility-based schedules, both Fig. 5 and Fig. 6 are optimal. Two adders and two multipliers are required.

(e) (4%)

To perform critical path scheduling, we prioritize the operation with the largest delay in its longest path to output. The ready lists and the chosen operations

for each ALU are as follows:

$$
\begin{aligned}
L_0 &= \{C_1, C_2, C_3\} && , A_1 : C_1, A_2 : C_2 \\
L_1 &= \{C_3, C_4\} && , A_1 : C_3 \\
L_3 &= \{C_4\} && , A_1 : C_4 \\
L_5 &= \{C_5\} && , A_1 : C_5
\end{aligned}
$$

The time chart is shown in Fig. 7

	0	1	2	3	4	5	6
A1	C1	C3		C4		C5	
A2	C2						

Fig. 7. The time chart of the scheduling result in problem 6(e).